```
LangChain Comprehensive Example The Cognitve Loop in Action
"""
LangChain Cognitive Loop Agent: Complete Implementation
Demonstrates: PERCEIVE → REASON → PLAN → ACT → LEARN cycle with Memory Systems

This agent implements a full cognitive architecture with:
1. PERCEPTION: Gathering and processing information
2. REASONING: Analyzing and understanding
3. PLANNING: Creating action strategies
4. ACTION: Executing plans
5. LEARNING: Updating knowledge and improving

Educational example showing realistic agent cognition.
"""

from langchain.agents import AgentExecutor, create_openai_functions_agent
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain.tools import Tool
from langchain.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain.memory import ConversationBufferMemory, ConversationSummaryMemory
from langchain.vectorstores import Chroma
from langchain.schema import Document, SystemMessage, HumanMessage, AIMessage
from typing import Dict, List, Any, Optional
from datetime import datetime
import json
import os

# ================================================================================
# PHASE 1: PERCEPTION - Information Gathering
# ================================================================================

class PerceptionModule:
    """
    Handles all input processing and environmental awareness.
    Simulates how agents perceive their environment.
    """

    def __init__(self):
        self.sensory_buffer = []  # Recent perceptions
        self.perception_history = []  # All perceptions over time

    def perceive_text_input(self, text: str) -> Dict[str, Any]:
        """Process textual input - like reading/listening"""
        perception = {
```

```
            "type": "text_input",
            "content": text,
            "timestamp": datetime.now().isoformat(),
            "word_count": len(text.split()),
            "contains_question": "?" in text,
            "sentiment": self._analyze_sentiment(text),
            "entities": self._extract_entities(text)
        }

        self.sensory_buffer.append(perception)
        self.perception_history.append(perception)

        print(f"\n{'='*60}")
        print("🔍 PERCEPTION PHASE")
        print(f"{'='*60}")
        print(f"Input received: {text}")
        print(f"Detected sentiment: {perception['sentiment']}")
        print(f"Entities found: {perception['entities']}")
        print(f"Question detected: {perception['contains_question']}")

        return perception

    def perceive_tool_output(self, tool_name: str, output: str) -> Dict[str, Any]:
        """Process tool/API responses - like sensing external data"""
        perception = {
            "type": "tool_output",
            "tool": tool_name,
            "content": output,
            "timestamp": datetime.now().isoformat(),
            "success": "error" not in output.lower()
        }

        self.sensory_buffer.append(perception)
        print(f"\n📡 Perceived tool output from {tool_name}")
        print(f"Success: {perception['success']}")

        return perception

    def get_current_context(self) -> str:
        """Get recent perceptions for context awareness"""
        recent = self.sensory_buffer[-5:]  # Last 5 perceptions
        context = "Recent perceptions:\n"
        for p in recent:
            context += f"- [{p['type']}] {p.get('content', '')[:100]}\n"
```

```
        return context

    def _analyze_sentiment(self, text: str) -> str:
        """Simple sentiment analysis"""
        positive_words = ["good", "great", "excellent", "happy", "love", "wonderful"]
        negative_words = ["bad", "terrible", "awful", "hate", "poor", "worst"]

        text_lower = text.lower()
        pos_count = sum(1 for word in positive_words if word in text_lower)
        neg_count = sum(1 for word in negative_words if word in text_lower)

        if pos_count > neg_count:
            return "positive"
        elif neg_count > pos_count:
            return "negative"
        return "neutral"

    def _extract_entities(self, text: str) -> List[str]:
        """Simple entity extraction (in production, use NER)"""
        # Simplified: just extract capitalized words as potential entities
        words = text.split()
        entities = [w.strip(",.!?") for w in words if w[0].isupper() and len(w) > 1]
        return entities[:5]  # Top 5


# ============================================================================
# PHASE 2: REASONING - Analysis and Understanding
# ============================================================================

class ReasoningModule:
    """
    Analyzes perceptions and draws conclusions.
    Implements different reasoning strategies.
    """

    def __init__(self, llm):
        self.llm = llm
        self.reasoning_history = []

    def analyze_situation(self, perception: Dict, memory_context: str) -> Dict[str, Any]:
        """Deep analysis of current situation"""

        print(f"\n{'='*60}")
        print("🧠 REASONING PHASE")
```

```
        print(f"{'='*60}")

        # Construct reasoning prompt
        reasoning_prompt = f"""
        Analyze the following situation carefully:

        Current Input: {perception.get('content', '')}
        Sentiment: {perception.get('sentiment', 'unknown')}
        Context from Memory: {memory_context}

        Provide analysis on:
        1. What is the user really asking or needing?
        2. What information do we have vs. what do we need?
        3. What are the key challenges or constraints?
        4. What patterns do you notice from past interactions?

        Be thorough and thoughtful.
        """

        response = self.llm.invoke([HumanMessage(content=reasoning_prompt)])
        analysis = response.content

        reasoning_result = {
            "analysis": analysis,
            "timestamp": datetime.now().isoformat(),
            "confidence": self._assess_confidence(analysis),
            "reasoning_type": "analytical"
        }

        self.reasoning_history.append(reasoning_result)

        print(f"\n Analysis:")
        print(analysis[:300] + "..." if len(analysis) > 300 else analysis)
        print(f"\nConfidence level: {reasoning_result['confidence']}")

        return reasoning_result

    def reason_about_approach(self, analysis: str, available_tools: List[str]) -> Dict[str, Any]:
        """Determine best approach given analysis and available tools"""

        prompt = f"""
        Based on this analysis:
        {analysis}
```

```
        Available tools: {', '.join(available_tools)}

        Determine:
        1. What approach should we take?
        2. Which tools should we use and in what order?
        3. What are potential risks or failure points?
        4. How can we validate our results?

        Think step by step.
        """

        response = self.llm.invoke([HumanMessage(content=prompt)])

        approach = {
            "strategy": response.content,
            "timestamp": datetime.now().isoformat(),
            "reasoning_type": "strategic"
        }

        print(f"\n🎯 Strategic reasoning:")
        print(approach["strategy"][:200] + "...")

        return approach

    def _assess_confidence(self, analysis: str) -> str:
        """Assess confidence in reasoning"""
        uncertainty_markers = ["might", "maybe", "possibly", "unclear", "uncertain"]
        confidence_markers = ["clearly", "definitely", "certainly", "obviously"]

        analysis_lower = analysis.lower()

        uncertain_count = sum(1 for marker in uncertainty_markers if marker in analysis_lower)
        confident_count = sum(1 for marker in confidence_markers if marker in analysis_lower)

        if confident_count > uncertain_count:
            return "high"
        elif uncertain_count > confident_count:
            return "low"
        return "medium"


# ================================================================================
# PHASE 3: PLANNING - Strategy Formation
# ================================================================================
```

```
class PlanningModule:
    """
    Creates action plans based on reasoning.
    Implements hierarchical and sequential planning.
    """

    def __init__(self, llm):
        self.llm = llm
        self.plans_created = []
        self.current_plan = None

    def create_plan(self, reasoning: Dict, goal: str) -> Dict[str, Any]:
        """Generate a detailed action plan"""

        print(f"\n{'='*60}")
        print(" 📋 PLANNING PHASE")
        print(f"{'='*60}")

        planning_prompt = f"""
        Create a detailed action plan:

        Goal: {goal}

        Analysis: {reasoning.get('analysis', '')}
        Strategy: {reasoning.get('strategy', '')}

        Create a step-by-step plan with:
        1. Specific actions to take
        2. Order of operations
        3. Expected outcomes for each step
        4. Fallback options if steps fail
        5. Success criteria

        Format as a clear, executable plan.
        """

        response = self.llm.invoke([HumanMessage(content=planning_prompt)])

        plan = {
            "goal": goal,
            "steps": self._parse_steps(response.content),
            "full_plan": response.content,
            "timestamp": datetime.now().isoformat(),
```

```
        "status": "ready",
        "estimated_difficulty": self._estimate_difficulty(response.content)
    }

    self.current_plan = plan
    self.plans_created.append(plan)

    print(f"\n📝 Plan created with {len(plan['steps'])} steps")
    print(f"Estimated difficulty: {plan['estimated_difficulty']}")
    print("\nPlan overview:")
    for i, step in enumerate(plan['steps'], 1):
        print(f"{i}. {step}")

    return plan

def adapt_plan(self, plan: Dict, execution_result: Dict) -> Dict[str, Any]:
    """Adapt plan based on execution feedback"""

    print(f"\n🔄 Adapting plan based on execution results...")

    if execution_result.get("success"):
        print("✓ Execution successful, plan remains valid")
        return plan

    print("⚠ Execution encountered issues, replanning...")

    adaptation_prompt = f"""
    Original plan:
    {plan['full_plan']}

    Execution result:
    {execution_result.get('details', 'Unknown issue')}

    Adapt the plan to address these issues.
    Provide an updated plan that accounts for what we learned.
    """

    response = self.llm.invoke([HumanMessage(content=adaptation_prompt)])

    adapted_plan = {
        **plan,
        "steps": self._parse_steps(response.content),
        "full_plan": response.content,
```

```
            "timestamp": datetime.now().isoformat(),
            "status": "adapted",
            "adaptation_reason": execution_result.get('details', '')
        }

        self.current_plan = adapted_plan

        return adapted_plan

    def _parse_steps(self, plan_text: str) -> List[str]:
        """Extract individual steps from plan"""
        lines = plan_text.split('\n')
        steps = []
        for line in lines:
            line = line.strip()
            if line and (line[0].isdigit() or line.startswith('-') or line.startswith('•')):
                # Remove numbering and bullet points
                step = line.lstrip('0123456789.-•) ').strip()
                if step:
                    steps.append(step)
        return steps[:10]  # Max 10 steps

    def _estimate_difficulty(self, plan: str) -> str:
        """Estimate plan complexity"""
        step_count = len(self._parse_steps(plan))
        complex_words = ["complex", "difficult", "challenging", "intricate"]

        has_complex = any(word in plan.lower() for word in complex_words)

        if step_count > 7 or has_complex:
            return "high"
        elif step_count > 4:
            return "medium"
        return "low"


# ============================================================================
# PHASE 4: ACTION - Plan Execution
# ============================================================================

class ActionModule:
    """
    Executes plans using available tools.
    Monitors execution and handles failures.
```

```
"""

def __init__(self, tools: List[Tool]):
    self.tools = {tool.name: tool for tool in tools}
    self.action_history = []

def execute_plan(self, plan: Dict, perception_module: PerceptionModule) -> Dict[str, Any]:
    """Execute the planned actions"""

    print(f"\n{'='*60}")
    print("⚡ ACTION PHASE")
    print(f"{'='*60}")

    results = []
    overall_success = True

    for i, step in enumerate(plan['steps'], 1):
        print(f"\nExecuting step {i}/{len(plan['steps'])}: {step[:60]}...")

        # Determine which tool to use
        tool_name = self._select_tool_for_step(step)

        if tool_name and tool_name in self.tools:
            try:
                tool = self.tools[tool_name]
                result = tool.func(step)

                # Perceive the tool output
                perception_module.perceive_tool_output(tool_name, result)

                step_result = {
                    "step": step,
                    "tool": tool_name,
                    "result": result,
                    "success": True,
                    "timestamp": datetime.now().isoformat()
                }

                print(f"✓ Success using {tool_name}")
                print(f"Result: {result[:100]}...")

            except Exception as e:
                step_result = {
                    "step": step,
```

```
                    "tool": tool_name,
                    "error": str(e),
                    "success": False,
                    "timestamp": datetime.now().isoformat()
                }
                overall_success = False
                print(f"✗ Failed: {str(e)}")
            else:
                # No tool needed, just log the action
                step_result = {
                    "step": step,
                    "tool": None,
                    "success": True,
                    "timestamp": datetime.now().isoformat()
                }
                print(f"→ Processed (no tool required)")

            results.append(step_result)
            self.action_history.append(step_result)

        execution_result = {
            "plan_id": plan.get("timestamp"),
            "steps_executed": len(results),
            "steps_successful": sum(1 for r in results if r["success"]),
            "overall_success": overall_success,
            "details": results,
            "timestamp": datetime.now().isoformat()
        }

        print(f"\n📊 Execution complete:")
        print(f"Success rate:
{execution_result['steps_successful']}/{execution_result['steps_executed']}")

        return execution_result

    def _select_tool_for_step(self, step: str) -> Optional[str]:
        """Determine which tool is needed for this step"""
        step_lower = step.lower()

        # Simple keyword matching (in production, use smarter selection)
        if any(word in step_lower for word in ["search", "find", "look up", "research"]):
            return "WebSearch"
        elif any(word in step_lower for word in ["calculate", "compute", "math"]):
            return "Calculator"
```

```
        elif any(word in step_lower for word in ["remember", "recall", "retrieve"]):
            return "MemorySearch"

        return None


# ============================================================================
# PHASE 5: LEARNING - Knowledge Update
# ============================================================================

class LearningModule:
    """
    Updates knowledge based on experiences.
    Implements multiple learning mechanisms.
    """

    def __init__(self, llm, embeddings):
        self.llm = llm
        self.embeddings = embeddings

        # Long-term semantic memory (vector store)
        self.semantic_memory = Chroma(
            collection_name="agent_knowledge",
            embedding_function=embeddings
        )

        # Episodic memory (specific experiences)
        self.episodic_memory = []

        # Procedural memory (learned strategies)
        self.procedural_memory = {
            "successful_strategies": [],
            "failed_strategies": [],
            "tool_effectiveness": {}
        }

        self.learning_events = []

    def learn_from_interaction(self, perception: Dict, reasoning: Dict,
                               plan: Dict, execution: Dict) -> Dict[str, Any]:
        """Learn from complete cognitive cycle"""

        print(f"\n{'='*60}")
        print("📚 LEARNING PHASE")
```

```
print(f"{'='*60}")

# 1. Episodic Learning: Store the experience
episode = {
    "perception": perception,
    "reasoning": reasoning,
    "plan": plan,
    "execution": execution,
    "timestamp": datetime.now().isoformat(),
    "success": execution.get("overall_success", False)
}

self.episodic_memory.append(episode)
print(f"\n💾 Stored episodic memory (total episodes: {len(self.episodic_memory)})")

# 2. Semantic Learning: Extract and store knowledge
knowledge_extracted = self._extract_knowledge(episode)
if knowledge_extracted:
    for knowledge_item in knowledge_extracted:
        doc = Document(
            page_content=knowledge_item["content"],
            metadata=knowledge_item["metadata"]
        )
        self.semantic_memory.add_documents([doc])
    print(f"✓ Added {len(knowledge_extracted)} items to semantic memory")

# 3. Procedural Learning: Update strategies
self._update_procedural_memory(plan, execution)

# 4. Meta-Learning: Learn about learning
learning_insights = self._meta_learning_analysis()

learning_result = {
    "episode_stored": True,
    "knowledge_items_added": len(knowledge_extracted) if knowledge_extracted else 0,
    "procedural_updates": self._get_procedural_summary(),
    "insights": learning_insights,
    "timestamp": datetime.now().isoformat()
}

self.learning_events.append(learning_result)

print(f"\n🎓 Learning complete:")
```

```
        print(f"Total experiences: {len(self.episodic_memory)}")
        print(f"Successful strategies learned:
{len(self.procedural_memory['successful_strategies'])}")
        print(f"\nKey insight: {learning_insights}")

        return learning_result

    def retrieve_relevant_knowledge(self, query: str, k: int = 3) -> str:
        """Retrieve relevant past knowledge"""
        print(f"\n🔍 Searching memory for: {query[:50]}...")

        if self.semantic_memory._collection.count() == 0:
            return "No prior knowledge found."

        results = self.semantic_memory.similarity_search(query, k=k)

        if not results:
            return "No relevant memories found."

        knowledge = "Relevant knowledge from memory:\n\n"
        for i, doc in enumerate(results, 1):
            knowledge += f"{i}. {doc.page_content}\n"
            knowledge += f"   (From: {doc.metadata.get('source', 'unknown')})\n\n"

        print(f"✓ Retrieved {len(results)} relevant memories")

        return knowledge

    def get_episodic_context(self, n_recent: int = 3) -> str:
        """Get recent experiences for context"""
        if not self.episodic_memory:
            return "No prior experiences."

        recent = self.episodic_memory[-n_recent:]
        context = "Recent experiences:\n\n"

        for i, episode in enumerate(recent, 1):
            success = "✓" if episode.get("success") else "✗"
            context += f"{i}. {success} {episode['perception'].get('content', '')[:60]}...\n"

        return context

    def _extract_knowledge(self, episode: Dict) -> List[Dict]:
```

```
    """Extract learnable knowledge from episode"""
    knowledge_items = []

    # Extract from successful actions
    if episode.get("success"):
        for action in episode['execution'].get('details', []):
            if action.get('success') and action.get('result'):
                knowledge_items.append({
                    "content": f"When {action['step']}, using {action.get('tool')} was effective. "
                               f"Result: {action['result'][:100]}",
                    "metadata": {
                        "source": "successful_action",
                        "timestamp": episode['timestamp'],
                        "tool": action.get('tool')
                    }
                })

    # Extract from reasoning
    if episode.get('reasoning', {}).get('analysis'):
        knowledge_items.append({
            "content": episode['reasoning']['analysis'][:500],
            "metadata": {
                "source": "reasoning_analysis",
                "timestamp": episode['timestamp']
            }
        })

    return knowledge_items

def _update_procedural_memory(self, plan: Dict, execution: Dict):
    """Update knowledge about what works"""
    strategy = {
        "plan": plan.get('full_plan', '')[:200],
        "success": execution.get('overall_success'),
        "timestamp": datetime.now().isoformat()
    }

    if execution.get('overall_success'):
        self.procedural_memory['successful_strategies'].append(strategy)
    else:
        self.procedural_memory['failed_strategies'].append(strategy)

    # Update tool effectiveness
    for action in execution.get('details', []):
```

```
            tool = action.get('tool')
            if tool:
                if tool not in self.procedural_memory['tool_effectiveness']:
                    self.procedural_memory['tool_effectiveness'][tool] = {
                        'successes': 0, 'failures': 0
                    }

                if action.get('success'):
                    self.procedural_memory['tool_effectiveness'][tool]['successes'] += 1
                else:
                    self.procedural_memory['tool_effectiveness'][tool]['failures'] += 1

    def _get_procedural_summary(self) -> str:
        """Summarize procedural knowledge"""
        tool_stats = self.procedural_memory['tool_effectiveness']
        if not tool_stats:
            return "No tool usage data yet"

        summary = "Tool effectiveness: "
        for tool, stats in tool_stats.items():
            total = stats['successes'] + stats['failures']
            rate = stats['successes'] / total if total > 0 else 0
            summary += f"{tool}({rate:.0%}), "

        return summary.rstrip(', ')

    def _meta_learning_analysis(self) -> str:
        """Learn about our own learning patterns"""
        if len(self.episodic_memory) < 2:
            return "Need more experience to identify patterns"

        recent_success_rate = sum(
            1 for e in self.episodic_memory[-5:] if e.get('success')
        ) / min(5, len(self.episodic_memory))

        if recent_success_rate > 0.7:
            return "Learning is progressing well - high recent success rate"
        elif recent_success_rate < 0.3:
            return "Need to adjust strategy - recent attempts struggling"
        else:
            return "Moderate success - continuing to learn and adapt"


# ================================================================================
```

```
# MEMORY SYSTEMS - Different types of agent memory
# ============================================================================

class MemorySystem:
    """
    Comprehensive memory architecture for the agent.
    Demonstrates different memory types and their roles.
    """

    def __init__(self, llm, embeddings):
        # 1. Working Memory (Short-term, immediate context)
        self.working_memory = ConversationBufferMemory(
            memory_key="chat_history",
            return_messages=True,
            output_key="output"
        )

        # 2. Short-term Summary Memory (Compressed recent context)
        self.summary_memory = ConversationSummaryMemory(
            llm=llm,
            memory_key="summary",
            return_messages=True
        )

        # 3. Long-term Semantic Memory (Facts and knowledge)
        self.semantic_memory = Chroma(
            collection_name="semantic_knowledge",
            embedding_function=embeddings
        )

        # 4. Episodic Memory (Specific experiences/events)
        self.episodic_memory = []

        # 5. Procedural Memory (How to do things)
        self.procedural_memory = {}

        print("\n" + "="*60)
        print("🧠 MEMORY SYSTEM INITIALIZED")
        print("="*60)
        print("Memory types active:")
        print("1. ✓ Working Memory (immediate context)")
        print("2. ✓ Summary Memory (compressed history)")
        print("3. ✓ Semantic Memory (factual knowledge)")
```

```
    print("4. ✓ Episodic Memory (specific experiences)")
    print("5. ✓ Procedural Memory (learned skills)")

def add_to_working_memory(self, input_text: str, output_text: str):
    """Add interaction to working memory"""
    self.working_memory.save_context(
        {"input": input_text},
        {"output": output_text}
    )

def get_working_memory_context(self) -> str:
    """Retrieve current working memory"""
    return str(self.working_memory.load_memory_variables({}).get('chat_history', []))

def consolidate_to_semantic(self, content: str, metadata: Dict):
    """Move important information to long-term semantic memory"""
    doc = Document(page_content=content, metadata=metadata)
    self.semantic_memory.add_documents([doc])
    print(f"💾 Consolidated to semantic memory: {content[:50]}...")

def retrieve_semantic(self, query: str, k: int = 3) -> List[Document]:
    """Retrieve from semantic memory"""
    if self.semantic_memory._collection.count() == 0:
        return []
    return self.semantic_memory.similarity_search(query, k=k)

def add_episodic(self, episode: Dict):
    """Store specific experience"""
    self.episodic_memory.append({
        **episode,
        "timestamp": datetime.now().isoformat()
    })

def get_similar_episodes(self, current_situation: str) -> List[Dict]:
    """Find similar past experiences"""
    # Simple similarity based on keywords (in production, use embeddings)
    current_words = set(current_situation.lower().split())

    similar = []
    for episode in self.episodic_memory:
        episode_words = set(str(episode).lower().split())
        similarity = len(current_words & episode_words) / len(current_words | episode_words)
        if similarity > 0.3:
```

```
            similar.append(episode)

        return similar[-3:]  # Return top 3 most recent similar episodes


# ================================================================================
# MAIN COGNITIVE AGENT - Integrating all modules
# ================================================================================

class CognitiveAgent:
    """
    Complete cognitive agent implementing full PERCEIVE-REASON-PLAN-ACT-LEARN cycle
    with comprehensive memory systems.
    """

    def __init__(self, api_key: str):
        # Initialize LLM and embeddings
        self.llm = ChatOpenAI(
            model="gpt-4",
            temperature=0.7,
            api_key=api_key
        )

        self.embeddings = OpenAIEmbeddings(api_key=api_key)

        # Initialize cognitive modules
        self.perception = PerceptionModule()
        self.reasoning = ReasoningModule(self.llm)
        self.planning = PlanningModule(self.llm)
        self.learning = LearningModule(self.llm, self.embeddings)

        # Initialize memory system
        self.memory = MemorySystem(self.llm, self.embeddings)

        # Define tools for action
        self.tools = self._create_tools()
        self.action = ActionModule(self.tools)

        # Cycle counter
        self.cycle_count = 0

    def _create_tools(self) -> List[Tool]:
        """Create tools for the agent"""
```

```
        def web_search(query: str) -> str:
            """Simulated web search"""
            return f"Search results for '{query}': Found relevant information about {query}. " \
                   f"Key points: [1] Important fact one, [2] Related concept, [3] Statistical data."

        def calculator(expression: str) -> str:
            """Calculator tool"""
            try:
                # Safe evaluation for demo (in production, use safer methods)
                result = eval(expression)
                return f"Calculation result: {result}"
            except Exception as e:
                return f"Calculation error: {str(e)}"

        def memory_search(query: str) -> str:
            """Search agent's memory"""
            return self.learning.retrieve_relevant_knowledge(query)

        return [
            Tool(
                name="WebSearch",
                func=web_search,
                description="Search the internet for current information. Use when you need external
knowledge."
            ),
            Tool(
                name="Calculator",
                func=calculator,
                description="Perform mathematical calculations. Input should be a valid mathematical
expression."
            ),
            Tool(
                name="MemorySearch",
                func=memory_search,
                description="Search the agent's memory for past knowledge and experiences."
            )
        ]

    def process_input(self, user_input: str) -> str:
        """
        Main cognitive loop: Process input through complete cycle
        """

        self.cycle_count += 1
```

```
print(f"\n\n{'#'*60}")
print(f"# COGNITIVE CYCLE #{self.cycle_count}")
print(f"{'#'*60}\n")

# ===== PHASE 1: PERCEIVE =====
perception_data = self.perception.perceive_text_input(user_input)

# ===== RETRIEVE FROM MEMORY =====
print(f"\n{'='*60}")
print(" MEMORY RETRIEVAL")
print(f"{'='*60}")

# Get working memory (recent conversation)
working_context = self.memory.get_working_memory_context()
print(f"\nWorking memory: {len(working_context)} characters")

# Get semantic memory (relevant knowledge)
semantic_knowledge = self.learning.retrieve_relevant_knowledge(user_input)

# Get episodic memory (similar past experiences)
similar_episodes = self.memory.get_similar_episodes(user_input)
episodic_context = f"Found {len(similar_episodes)} similar past experiences"
print(f"Episodic memory: {episodic_context}")

# Combine all memory sources
full_memory_context = f"""
Recent conversation: {working_context[:200]}

Relevant knowledge: {semantic_knowledge[:300]}

Similar experiences: {episodic_context}
"""

# ===== PHASE 2: REASON =====
reasoning_result = self.reasoning.analyze_situation(
    perception_data,
    full_memory_context
)

# Strategic reasoning about approach
available_tools = [tool.name for tool in self.tools]
approach = self.reasoning.reason_about_approach(
    reasoning_result['analysis'],
```

```
    available_tools
)

# ===== PHASE 3: PLAN =====
plan = self.planning.create_plan(
    {**reasoning_result, **approach},
    goal=user_input
)

# ===== PHASE 4: ACT =====
execution_result = self.action.execute_plan(plan, self.perception)

# Check if plan needs adaptation
if not execution_result.get('overall_success'):
    plan = self.planning.adapt_plan(plan, execution_result)
    # Re-execute adapted plan
    execution_result = self.action.execute_plan(plan, self.perception)

# ===== PHASE 5: LEARN =====
learning_result = self.learning.learn_from_interaction(
    perception=perception_data,
    reasoning=reasoning_result,
    plan=plan,
    execution=execution_result
)

# ===== UPDATE MEMORY SYSTEMS =====
print(f"\n{'='*60}")
print("💾 UPDATING MEMORY SYSTEMS")
print(f"{'='*60}")

# Generate response to user
final_response = self._generate_response(
    perception_data,
    reasoning_result,
    execution_result
)

# Update working memory
self.memory.add_to_working_memory(user_input, final_response)
print("✓ Updated working memory")

# Consolidate important information to semantic memory
if execution_result.get('overall_success'):
```

```
            self.memory.consolidate_to_semantic(
                content=f"Q: {user_input}\nA: {final_response[:200]}",
                metadata={
                    "type": "qa_pair",
                    "success": True,
                    "timestamp": datetime.now().isoformat()
                }
            )
            print("✓ Consolidated to semantic memory")

        # Add complete experience to episodic memory
        self.memory.add_episodic({
            "input": user_input,
            "response": final_response,
            "success": execution_result.get('overall_success'),
            "cycle": self.cycle_count
        })
        print("✓ Stored episodic memory")

        # Print cycle summary
        self._print_cycle_summary(learning_result)

        return final_response

    def _generate_response(self, perception: Dict, reasoning: Dict,
                           execution: Dict) -> str:
        """Generate final response to user"""

        prompt = f"""
        Based on the following cognitive process, provide a helpful response to the user:

        User input: {perception.get('content')}

        Analysis performed: {reasoning.get('analysis', '')[:200]}

        Actions taken: {len(execution.get('details', []))} steps executed
        Success rate: {execution.get('steps_successful', 0)}/{execution.get('steps_executed', 0)}

        Execution details:
        {self._format_execution_details(execution)}

        Provide a clear, helpful response that:
        1. Directly answers the user's question or need
        2. Explains what you did and why
```

```
    3. Includes relevant information from your actions
    4. Is friendly and conversational

    Keep it concise but complete.
    """

    response = self.llm.invoke([HumanMessage(content=prompt)])
    return response.content

def _format_execution_details(self, execution: Dict) -> str:
    """Format execution results for response generation"""
    details = execution.get('details', [])
    formatted = ""

    for i, detail in enumerate(details[:5], 1):  # Top 5 steps
        if detail.get('success'):
            formatted += f"{i}. {detail.get('step', '')[:60]} - ✓\n"
            if detail.get('result'):
                formatted += f"   Result: {detail['result'][:100]}\n"

    return formatted

def _print_cycle_summary(self, learning_result: Dict):
    """Print summary of cognitive cycle"""

    print(f"\n{'='*60}")
    print("📊 COGNITIVE CYCLE SUMMARY")
    print(f"{'='*60}")
    print(f"Cycle number: {self.cycle_count}")
    print(f"Total perceptions: {len(self.perception.perception_history)}")
    print(f"Total episodes in memory: {len(self.memory.episodic_memory)}")
    print(f"Semantic knowledge items: {self.learning.semantic_memory._collection.count()}")
    print(f"Learning insight: {learning_result.get('insights', 'N/A')}")
    print(f"{'='*60}\n")

def get_memory_statistics(self) -> Dict[str, Any]:
    """Get comprehensive memory statistics"""

    stats = {
        "working_memory_size": len(self.memory.get_working_memory_context()),
        "episodic_memories": len(self.memory.episodic_memory),
        "semantic_knowledge_items": self.learning.semantic_memory._collection.count(),
        "total_perceptions": len(self.perception.perception_history),
        "total_reasoning_events": len(self.reasoning.reasoning_history),
```

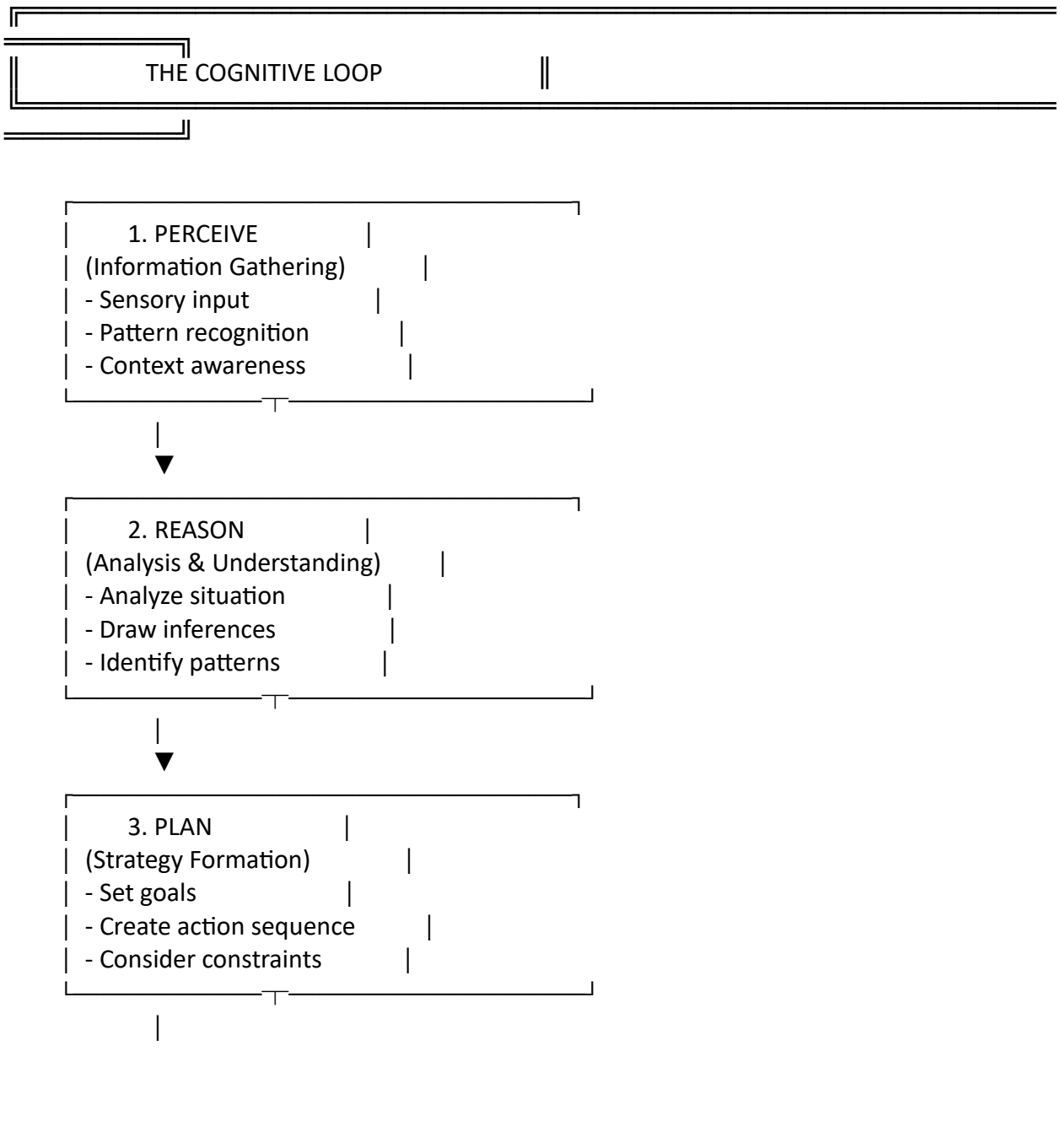
```
            "total_plans_created": len(self.planning.plans_created),
            "total_actions_taken": len(self.action.action_history),
            "learning_events": len(self.learning.learning_events),
            "successful_strategies": len(self.learning.procedural_memory['successful_strategies']),
            "failed_strategies": len(self.learning.procedural_memory['failed_strategies']),
            "tool_effectiveness": self.learning.procedural_memory['tool_effectiveness'],
            "total_cycles": self.cycle_count
        }

        return stats

    def reflect_on_experience(self) -> str:
        """Meta-cognitive reflection on agent's experience"""

        stats = self.get_memory_statistics()

        reflection_prompt = f"""
        Reflect on your learning and experience as an AI agent:

        Statistics:
        - Total cognitive cycles: {stats['total_cycles']}
        - Episodic memories: {stats['episodic_memories']}
        - Semantic knowledge items: {stats['semantic_knowledge_items']}
        - Successful strategies learned: {stats['successful_strategies']}
        - Failed strategies learned: {stats['failed_strategies']}

        Tool effectiveness:
        {json.dumps(stats['tool_effectiveness'], indent=2)}

        Provide insights on:
        1. What patterns have you noticed in the tasks you've handled?
        2. What strategies work best?
        3. What areas need improvement?
        4. What have you learned about learning itself?

        Be thoughtful and analytical.
        """

        response = self.llm.invoke([HumanMessage(content=reflection_prompt)])

        return response.content


# ================================================================================
```

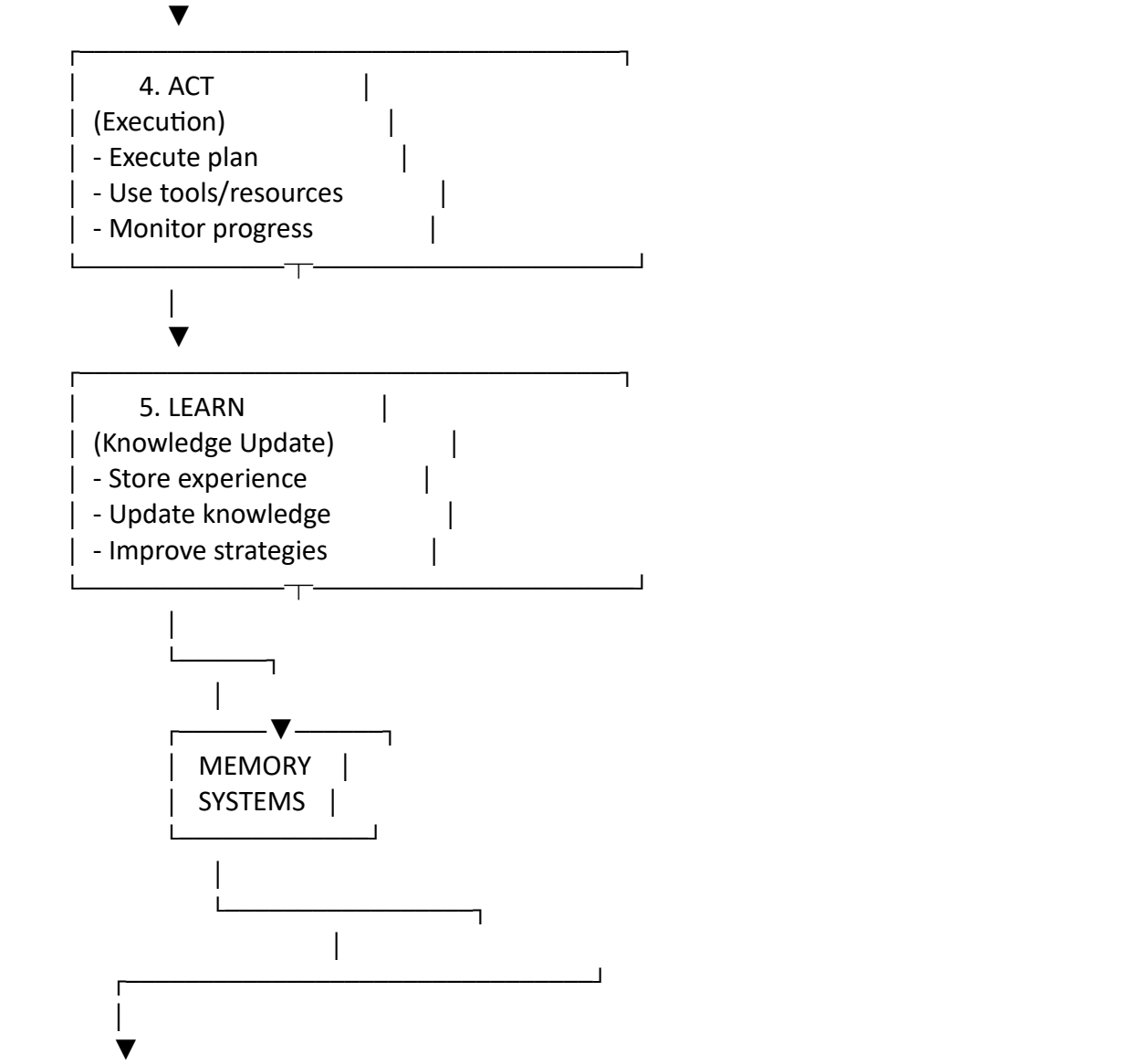
```
# DEMONSTRATION AND EDUCATIONAL EXAMPLES
# ============================================================================

def demonstrate_cognitive_loop():
    """
    Educational demonstration of the complete cognitive loop.
    Shows each phase in action with clear explanations.
    """

    print("\n" + "="*70)
    print("  COGNITIVE AGENT DEMONSTRATION")
    print("  Illustrating: PERCEIVE → REASON → PLAN → ACT → LEARN")
    print("="*70)

    # Initialize agent
    agent = CognitiveAgent(api_key="your-api-key-here")

    # Example 1: Simple information retrieval
    print("\n\n" + "#"*70)
    print("# EXAMPLE 1: Information Retrieval & Learning")
    print("#"*70)

    response1 = agent.process_input(
        "What are the key benefits of AI agents in customer service?"
    )

    print(f"\n{'='*60}")
    print("FINAL RESPONSE TO USER:")
    print(f"{'='*60}")
    print(response1)

    # Example 2: Problem solving with calculation
    print("\n\n" + "#"*70)
    print("# EXAMPLE 2: Problem Solving with Tools")
    print("#"*70)

    response2 = agent.process_input(
        "If a company has 5 customer service agents handling 200 tickets per day, "
        "and they want to reduce response time by 40% using AI, how many AI agents "
        "would be equivalent to one human agent if AI is 3x faster?"
    )

    print(f"\n{'='*60}")
    print("FINAL RESPONSE TO USER:")
```

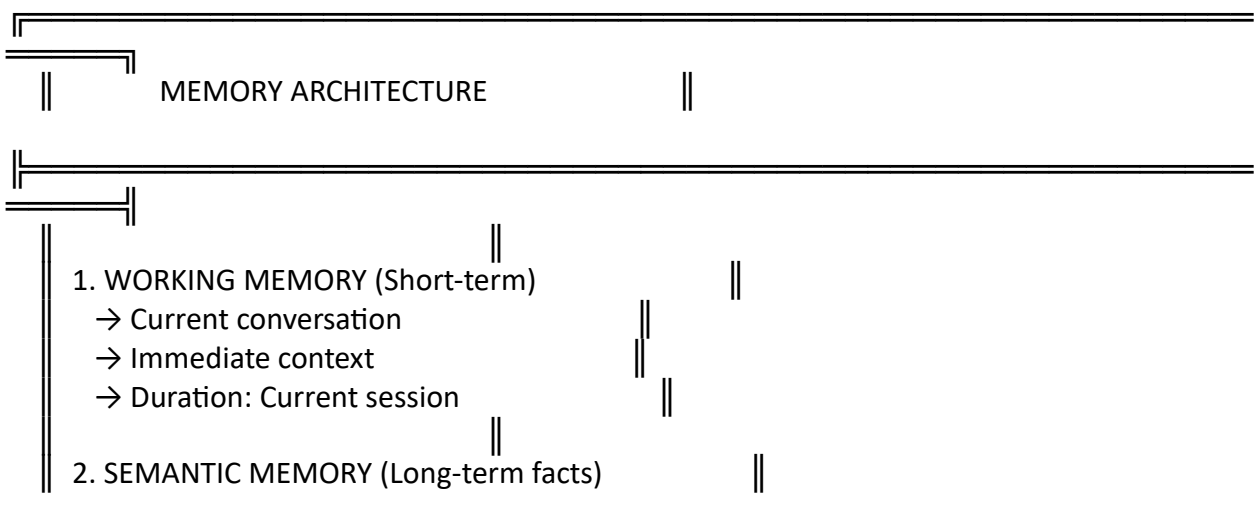
```
print(f"{'='*60}")
print(response2)

# Example 3: Using memory from previous interactions
print("\n\n" + "#"*70)
print("# EXAMPLE 3: Memory Utilization")
print("#"*70)

response3 = agent.process_input(
    "Based on what we discussed earlier about AI agents, "
    "what would be the ROI calculation framework?"
)

print(f"\n{'='*60}")
print("FINAL RESPONSE TO USER:")
print(f"{'='*60}")
print(response3)

# Example 4: Complex multi-step reasoning
print("\n\n" + "#"*70)
print("# EXAMPLE 4: Complex Multi-Step Reasoning")
print("#"*70)

response4 = agent.process_input(
    "Create a step-by-step implementation plan for deploying "
    "AI agents in a small business with 20 employees"
)

print(f"\n{'='*60}")
print("FINAL RESPONSE TO USER:")
print(f"{'='*60}")
print(response4)

# Show memory statistics
print("\n\n" + "="*70)
print("  MEMORY SYSTEMS ANALYSIS")
print("="*70)

stats = agent.get_memory_statistics()

print("\nMemory Statistics:")
print(f"  Working Memory Size: {stats['working_memory_size']} chars")
print(f"  Episodic Memories: {stats['episodic_memories']}")
print(f"  Semantic Knowledge: {stats['semantic_knowledge_items']} items")
```

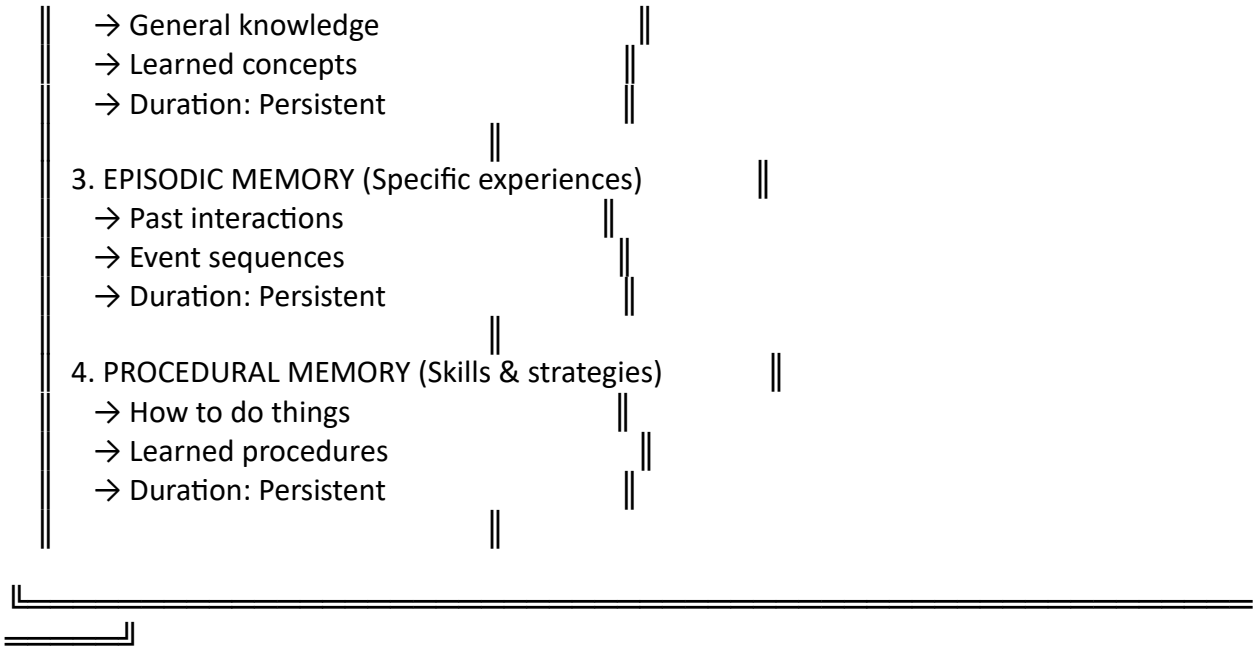
```
print(f"  Total Perceptions: {stats['total_perceptions']}")
print(f"  Plans Created: {stats['total_plans_created']}")
print(f"  Actions Executed: {stats['total_actions_taken']}")
print(f"  Cognitive Cycles: {stats['total_cycles']}")

print("\nLearning Progress:")
print(f"  Successful Strategies: {stats['successful_strategies']}")
print(f"  Failed Strategies: {stats['failed_strategies']}")
print(f"  Learning Events: {stats['learning_events']}")

if stats['tool_effectiveness']:
    print("\nTool Effectiveness:")
    for tool, effectiveness in stats['tool_effectiveness'].items():
        total = effectiveness['successes'] + effectiveness['failures']
        success_rate = effectiveness['successes'] / total if total > 0 else 0
        print(f"  {tool}: {success_rate:.1%} success rate "
              f"({effectiveness['successes']}/{total} attempts)")

# Agent reflection
print("\n\n" + "="*70)
print("  AGENT SELF-REFLECTION")
print("="*70)

reflection = agent.reflect_on_experience()
print(f"\n{reflection}")

print("\n\n" + "="*70)
print("  DEMONSTRATION COMPLETE")
print("="*70)
print("\nKey Takeaways:")
print("1. ✓ PERCEPTION: Agent processes and interprets inputs")
print("2. ✓ REASONING: Agent analyzes situations and determines approaches")
print("3. ✓ PLANNING: Agent creates structured action plans")
print("4. ✓ ACTION: Agent executes plans using available tools")
print("5. ✓ LEARNING: Agent updates knowledge and improves over time")
print("\nMemory Systems:")
print("• Working Memory: Maintains current conversation context")
print("• Semantic Memory: Stores factual knowledge long-term")
print("• Episodic Memory: Remembers specific experiences")
print("• Procedural Memory: Learns what strategies work")
print("\nThe agent continuously improves through this cognitive loop!")
```

```
def educational_breakdown():
    """
    Step-by-step educational breakdown of each cognitive phase.
    Perfect for classroom teaching.
    """

    print("\n" + "="*70)
    print("  EDUCATIONAL BREAKDOWN: THE COGNITIVE LOOP")
    print("="*70)

    print("""
```




4. ACT
(Execution)
- Execute plan
- Use tools/resources
- Monitor progress
5. LEARN
(Knowledge Update)
- Store experience
- Update knowledge
- Improve strategies
MEMORY
SYSTEMS


MEMORY ARCHITECTURE
1. WORKING MEMORY (Short-term)
→ Current conversation
→ Immediate context
→ Duration: Current session
2. SEMANTIC MEMORY (Long-term facts)



KEY INSIGHTS:

1. PERCEPTION is not passive - the agent actively interprets
   and structures incoming information.

2. REASONING involves multiple types:
   - Analytical: Understanding the situation
   - Strategic: Determining the best approach
   - Causal: Understanding cause-effect relationships

3. PLANNING is hierarchical:
   - High-level goals
   - Mid-level strategies
   - Low-level actions

4. ACTION includes:
   - Tool selection
   - Execution monitoring
   - Error handling
   - Adaptation

5. LEARNING happens at multiple levels:
   - Episodic: Remembering what happened
   - Semantic: Extracting general knowledge
   - Procedural: Learning what works

```
   - Meta-learning: Learning about learning

6. MEMORY is not a single system but multiple specialized systems
   working together to provide different types of information.

The cognitive loop is CONTINUOUS - each cycle informs the next,
creating an agent that improves over time through experience.
    """)


# ============================================================================
# RUN THE DEMONSTRATIONS
# ============================================================================

if __name__ == "__main__":
    print("\n" + "="*70)
    print("  COGNITIVE AGENT WITH COMPLETE MEMORY SYSTEMS")
    print("  Educational Implementation for Teaching AI Agents")
    print("="*70)

    # Show educational breakdown first
    educational_breakdown()

    # Wait for user to continue
    input("\n\nPress Enter to see the cognitive loop in action...")

    # Run the full demonstration
    demonstrate_cognitive_loop()

    print("\n\n" + "="*70)
    print("  END OF DEMONSTRATION")
    print("="*70)
    print("\nThis example demonstrates:")
    print("✓ Complete cognitive loop implementation")
    print("✓ All five cognitive phases working together")
    print("✓ Multiple memory systems and their roles")
    print("✓ Learning and improvement over time")
    print("✓ Realistic agent behavior patterns")
    print("\nPerfect for teaching students about agent architectures!")
    print("="*70)
```